

Обработка данных в JavaScript

Занятие 8: Data Processing — **map / filter / reduce**

Инструктор — Андрей Помелов





Для чего нужны методы массивов?

При работе с данными нам часто нужно **преобразовывать**, **фильтровать** и **агрегировать** массивы. Ручные циклы `for` работают, но делают код громоздким и трудночитаемым.

✗ Без методов

Ручные циклы, сложный код

✓ С методами

Декларативно, чисто, читаемо

Методы массивов — обзор

JavaScript предоставляет встроенные методы, которые принимают **функцию обратного вызова** и применяют её к каждому элементу массива. Это основа функционального подхода к обработке данных.



1

map

Преобразование элементов

2

filter

Отбор по условию

3

reduce

Свёртка к значению

4

Другие

find, some, every...



→ МЕТОД

map()

Создаёт **новый массив**, преобразуя каждый элемент исходного по заданному правилу. Исходный массив **не изменяется**.

```
const nums = [1, 2, 3];  
const doubled = nums.map(n => n  
  * 2);  
// [2, 4, 6]
```

 Всегда возвращает массив той же длины.

🔍 МЕТОД

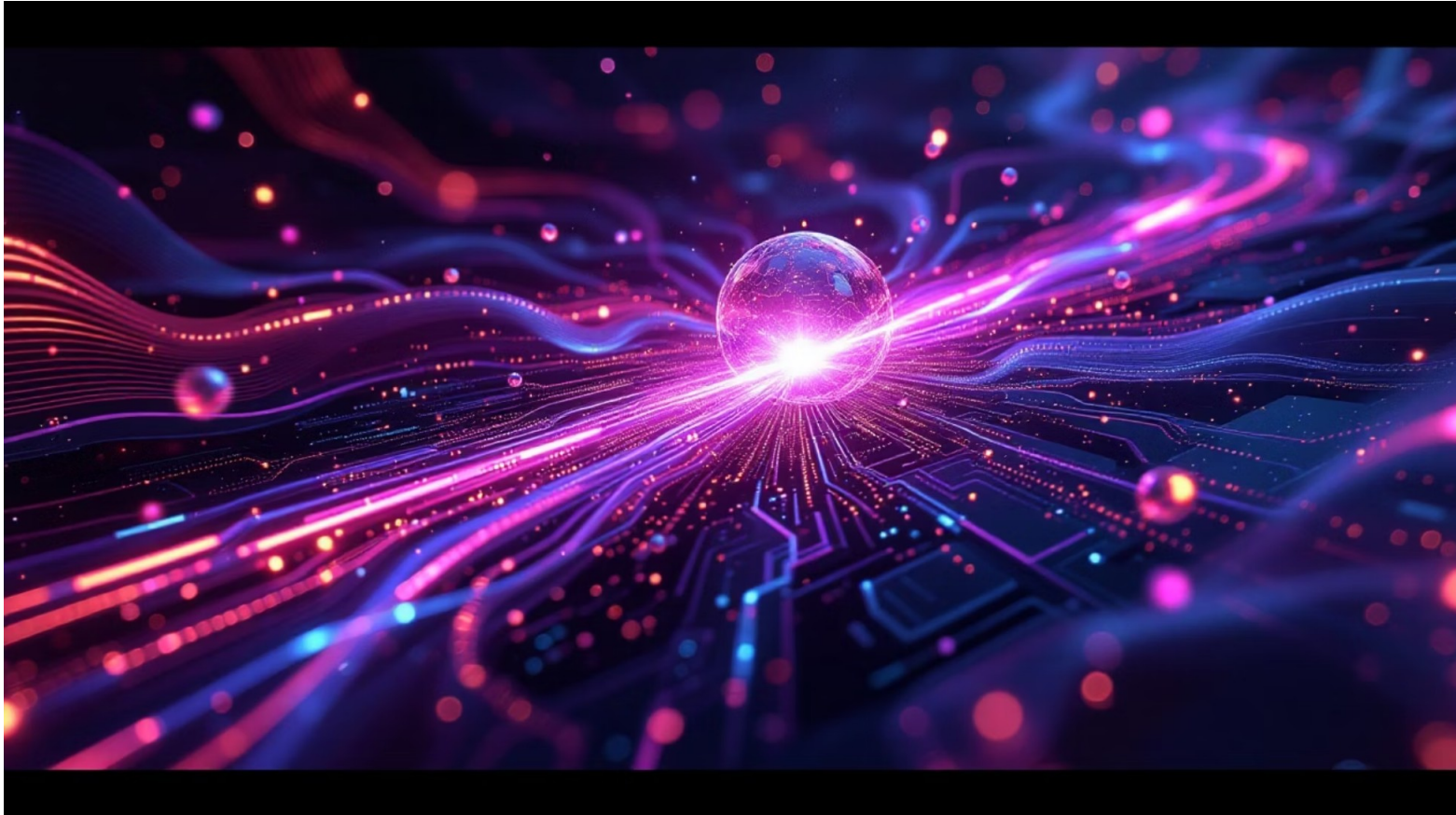
filter()

Возвращает **новый массив**, содержащий только те элементы, для которых функция-условие вернула `true`. Элементы не изменяются.

```
const ages = [14, 22, 17, 30];  
const adults = ages.filter(a =>  
  a >= 18);  
// [22, 30]
```

📘 Длина результата может быть меньше исходной.





⚙️ МЕТОД

reduce()

«Сворачивает» весь массив к **одному значению**: числу, строке, объекту. Принимает аккумулятор и текущий элемент.

```
const nums = [1, 2, 3, 4];  
const sum = nums.reduce(  
  (acc, n) => acc + n, 0  
);  
// 10
```

⚠️ Не забывайте указывать начальное значение аккумулятора.

Дополнительные методы

find / findIndex

Находит первый элемент (или его индекс), удовлетворяющий условию.

some / every

`some` — хотя бы один; `every` — все элементы соответствуют условию.

sort

Сортирует элементы массива. Принимает функцию-компаратор для точного управления порядком.

forEach

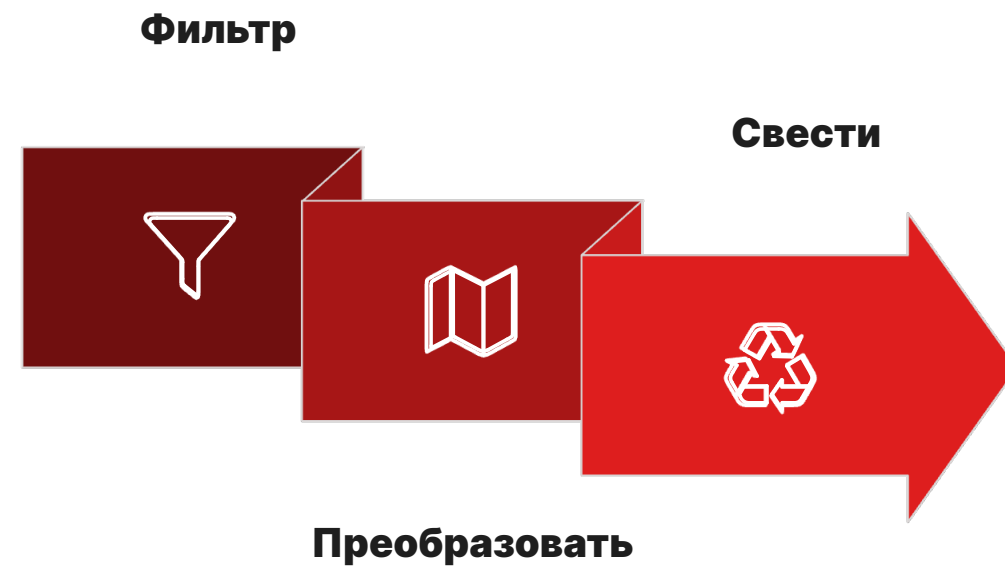
Перебирает элементы ради побочных эффектов. В отличие от `map`, не возвращает новый массив.



Цепочки методов

Методы можно **объединять последовательно** — результат одного становится входом следующего. Это делает обработку данных выразительной и читаемой.

```
const result = users
  .filter(u => u.active)      // только активные
  .map(u => u.salary)         // берём зарплату
  .reduce((sum, s) => sum + s, 0); // суммируем
```



Каждый шаг цепочки делает одно конкретное дело — код остаётся понятным и легко тестируемым.

Итог занятия

1 Декларативный стиль

Методы массивов описывают *что* нужно сделать, а не *как* — код становится лаконичнее.

2 Три ключевых метода

map — преобразование, **filter** — отбор, **reduce** — свёртка. Освойте их в первую очередь.

3 Цепочки — ваш инструмент

Комбинируйте методы для построения понятных многошаговых преобразований данных.